

Guia Completo: Modernização de Java 7 para Java 21

Material de estudo para entrevistas e refatoração de código legado

1. Código Original (Java 7)

```
List<Motorista> motoristas = getMotoristasDaEmpresa();
List<Motorista> habilitados = new ArrayList<>();

for(Motorista m : motoristas) {
    if(m.isChnAtiva()) {
        habilitados.add(m);
    }
}

Collections.sort(habilitados, new Comparator<Motorista>() {
    @Override
    public int compare(Motorista m1, Motorista m2) {
        return Integer.compare(m1.getAnosEmpresa(), m2.getAnosEmpresa());
    }
});
```

Problemas da abordagem antiga

- Muito código boilerplate.
- Mistura de regras de negócio com detalhes de implementação.
- Comparator anônimo difícil de reutilizar.
- Manipulação manual de listas.
- Menor legibilidade para equipes modernas.

2. Código Modernizado (Java 21)

```
List<Motorista> habilitados = getMotoristasDaEmpresa().stream()  
    .filter(Motorista::isChnAtiva)  
    .sorted(Comparator.comparingInt(Motorista::getAnosEmpresa))  
    .toList();
```

Leitura declarativa

A Stream descreve o que deve acontecer com os dados, e não como percorrer manualmente a coleção. Isso produz código mais expressivo e mais fácil de manter.

Fluxo de execução

```
Motoristas  
↓  
Filtro CNH ativa  
↓  
Ordenação por tempo de empresa  
↓  
Lista final
```

3. Explicação Detalhada das Mudanças

Stream API

Substitui laços explícitos por uma pipeline de processamento de dados.

Method References

Motorista::isChnAtiva é mais legível do que expressões anônimas equivalentes.

Comparator.comparingInt

Reduz código repetitivo e evita erros em implementações manuais de compare().

toList()

Produz uma lista não modificável, aumentando a segurança do resultado.

Programação Declarativa

O código descreve a intenção de negócio em vez dos detalhes de iteração.

4. Discussão sobre Optional e Regras de Negócio

Durante a conversa foi discutido como evoluir a Stream para lidar com um Optional de Seguro associado ao motorista.

Forma recomendada de pensar

```
Todos os motoristas
↓
CNH ativa?
↓
Possui seguro?
↓
Seguro está ativo?
↓
Ordenar por tempo de empresa
↓
Resultado final
```

Cada filter() deve representar uma única regra de negócio. Isso facilita testes, manutenção e futuras evoluções.

Benefícios do Optional

- Explicita ausência de valor.
- Reduz NullPointerException.
- Força o desenvolvedor a tratar casos de ausência.
- Torna o domínio mais claro.

5. Perguntas Comuns em Entrevistas Java Sênior

Por que Streams são melhores que loops?

Porque expressam intenção, reduzem boilerplate e permitem composição de operações.

Quando evitar Streams?

Em fluxos extremamente complexos ou quando a legibilidade fica pior que a versão imperativa.

Qual a diferença entre `map()` e `filter()`?

`filter` remove elementos; `map` transforma elementos.

Por que usar `Optional`?

Para modelar ausência de valor de forma explícita.

Qual a vantagem de `Comparator.comparingInt()`?

Maior legibilidade e menor chance de erros na comparação.

6. Resumo para Revisão Rápida

Java 7 → Loops + Collections.sort + Comparator anônimo

Java 21 → Stream + Method Reference + Comparator.comparingInt + toList()

Mentalidade recomendada:

- Cada filter() = uma regra de negócio.
- Optional representa ausência de valor.
- Código declarativo > código imperativo quando aumenta a legibilidade.
- Streams favorecem manutenção e evolução do sistema.